Kavach: Deterministic-Primary Detection for LLM Agent Tool Calls

A Rearchitecture, a Real-Trajectory Validation, and Negative Results

Anonymous Author(s)

Abstract

LLM agents increasingly act in the world through tool calls, but the guardrails deployed to secure them overwhelmingly evaluate an agent's language — its prompt, its chain-of-thought, its output — rather than the resolved arguments of the action it is about to take. We present Kavach, a runtime monitor that intercepts every tool call at the moment its arguments are resolved, before any side effect occurs. Kavach's detection is organized as a parliament of four domain-specialized ministers — VAULT (credentials), EXECUTOR (dangerous code), CHANNEL (exfiltration), and NAVIGATOR (unauthorized action) — whose verdicts a deterministic speaker combines into Block, Escalate, or Allow. We make four contributions. First, a methodology finding: four independent lines of evidence — a lexical-inflation bug, a dense fine-tune that traded one failure mode for a worse one, a corpus-coverage gap, and a proposed fifth minister formally falsified before being built (near-zero cosine-similarity deltas between paired legitimate/malicious calls) — together show that deterministic parsing over an action's resolved arguments outperforms cosine similarity over its semantic embedding when the attack signal is structural. We moved three of Kavach's four detectors to deterministic-primary detection on this evidence. Second, a deployment with real numbers: deterministic-primary detection with escalate-only cosine triage reaches 74.4%/90.1% on a 519-case reference corpus with zero new benign false positives, 544/544 on InjecAgent's data-stealing corpus (direct-harm remains an open gap: 0% strict recall, §4.7), and a session-level provenance mechanism that correctly escalates 5/5 structurally applicable AgentDojo attacks — verified against both the benchmark's ground-truth sequence and AgentDojo's own full recorded model trajectories — with zero false blocks on 22 real benign sessions. Third, a large-scale real-trajectory validation: replaying all 629 real recorded GPT-4o attack trajectories AgentDojo has published, Kavach fires on 74.2%, and we trace every apparent miss to its actual tool-call content, distinguishing benchmark-labeling artifacts from genuine, precisely categorized gaps. Fourth, negative results reported in full: three independently designed heuristics for a session-level authorization layer, an abandoned deterministic swap attempt, and an independent generalization audit against a fictional vocabulary that establishes precisely which detection mechanisms transfer past benchmark vocabulary and which do not. Kavach needs no GPU and is fully reproducible offline.

1 Introduction

LLM agents increasingly act in the world through tool calls: reading files, sending money, messaging other people, executing code. The guardrails deployed to secure this behavior overwhelmingly evaluate the agent's *language* — its input prompt, its chain-of-thought, its output text — rather than the *resolved arguments* of the action it is about to take. This is a real gap: a tool call to `send_money` with an attacker-substituted recipient IBAN, or to a credential-harvesting shell one-liner, is often indistinguishable from a benign call at the language level and only becomes distinguishable once its concrete arguments are inspected. Kavach is a runtime monitor that closes this gap by intercepting every tool call at the moment its arguments are resolved and scoring the resulting action — not the prompt, not the plan, the resolved call itself — before any side effect occurs.

This paper is substantially a report on a design decision and the evidence that produced it: we built Kavach's first version as a semantic, embedding-based detector — a parliament of domain-specialized ministers, each retrieving against a curated corpus of attack-pattern descriptions – and over the course of developing and stress-testing it, accumulated four independent lines of evidence that this design was the wrong primary mechanism for three of its four detection domains, while remaining the right mechanism for the fourth. We report both the evidence and the resulting rearchitecture, including two attempts that did not work, because a paper that reports only the final, working configuration understates how the field should update on evidence like this: not every detection problem shaped by an attacker's resolved arguments is best solved by comparing embeddings.

Contributions. **First, a methodology finding: deterministic parsing beats cosine similarity when the attack signal is structural (§3).** A lexical-inflation bug, a dense fine-tune that traded one failure mode for a worse one, a corpus-coverage gap that generalized past its original motivating case, and a proposed fifth minister that we formally falsified before building — paired legitimate/malicious tool calls that differ only in argument content show near-zero cosine similarity deltas (0.003–0.024) — together make the case that credential-path shapes, network-fetch-to-disk patterns, and destination-value provenance are answered more reliably by deterministic parsing over an action's resolved arguments than by any refinement of embedding similarity over its semantic content. We moved three of Kavach's four ministers (VAULT, EXECUTOR, CHANNEL) to deterministic-primary detection on this evidence, keeping cosine similarity

only as an escalate-only triage layer over the deterministic residual.

Second, a deployment with real detection numbers (§3, §4). InjecAgent and AgentDojo, the standard benchmarks we also evaluate against below, almost entirely miss VAULT and EXECUTOR — both stress CHANNEL's mechanisms specifically, a gap we found live while validating this rearchitecture, not one we anticipated. We built Kavach-PB (§4.2), a composite benchmark that closes this gap by reusing real, externally-validated attack data (AtomicRedTeam, GTFOBins) under one runner and one per-minister scoring convention, rather than authoring new synthetic attacks that risk being unconsciously shaped to be catchable by the system being evaluated. On Kavach-PB's 519-case reference corpus, deterministic-primary detection with cosine triage reaches VAULT 74.4% and EXECUTOR 90.1%, stable under a held-out split, with zero new benign false positives across two independent benign populations. CHANNEL's session-level provenance mechanism — tracking whether a consequential call's argument values trace to the user's own instruction or to an earlier, untrusted read this session — correctly escalates 5/22 real benign multi-step sessions with zero false blocks, and 5/5 structurally applicable AgentDojo attack cases, verified independently against both the benchmark's minimal ground-truth call sequence and AgentDojo's own full recorded model trajectories for the same cases. On InjecAgent's data-stealing corpus, the deterministic pipeline reaches 544/544 detection; the direct-harm corpus remains an open gap (0% strict recall, §4.7).

Third, a large-scale real-trajectory validation (§4.6). Beyond benchmark replay, we validated Kavach against all 629 real recorded GPT-4o attack trajectories published by AgentDojo — actual model-generated tool-call sequences with real tool outputs, not a benchmark's minimal answer key. Kavach fires on 467/629 (74.2%) of these. Tracing every apparent miss to its actual tool-call content shows 131 of the 162 misses reflect the model never executing any consequential action in connection with the injected instruction at all — an artifact of AgentDojo's own success-labeling convention, not a detection gap — leaving 19 genuine misses, which we categorize precisely by mechanism (five are a scoped CHANNEL destination-tool gap; fourteen have no destination value to trace at all, and are a different kind of problem entirely).

Fourth, honestly reported negative results (§6). NAVIGATOR, the one minister we did not move to deterministic-primary detection, is reported in full: a first attempt at a direct deterministic swap showed zero net discrimination gain and was abandoned at its own go/no-go gate; a second attempt reframed the minister's question as session-level authorization rather than similarity, and we report three independently designed heuristics for the resulting hard-constraint layer, all of which failed a frozen discriminability criterion set before measurement, for three distinct and informative reasons. An independent generalization audit, run against a fictional agent vocabulary sharing no surface term with any benchmark Kavach was tuned against, further establishes precisely which of the four ministers' mechanisms generalize past benchmark-specific vocabulary and which do not — EXECUTOR generalizes cleanly (with one structural false positive found and fixed during the audit itself); VAULT generalizes except where its mechanism is an inherent literal-name allowlist; CHANNEL's escalation path generalizes while its hard-block path does not; NAVIGATOR's cosine detector does not transfer to novel vocabulary at all. We report these boundaries because knowing precisely where each mechanism does and does not generalize is what distinguishes a system whose limits are known from one whose limits have simply not yet been found.

Scope and non-claims. We do not claim Kavach prevents every class of agent-side attack. Its deterministic rules are authored against published taxonomies and known attack corpora; a genuinely novel technique outside both may not be caught, and the generalization audit (§6) characterizes this precisely rather than leaving it as an assumption. NAVIGATOR's authorization question — was this specific action derivable from what the user actually asked for — remains open past a validated taxonomy classifier; we report the attempts and their failure modes rather than a completed mechanism. We do not propose Kavach as a replacement for input-time or output-time guardrails; it enforces at a different boundary along the agent's execution lifecycle, and a comprehensive deployment runs more than one.

Reproducibility. The parliament service, the deterministic prefilters and their test suites, the reference-corpus evaluation harness, and the real-trajectory replay scripts used for every number in §4 are shipped with the paper and run entirely offline on CPU-only hardware.

2 Background

2.1 The agent infrastructure attack surface

Modern LLM agents are composite systems: a language model provides the reasoning surface, while the harness provides tool dispatch, context management, multi-turn state, and persistence. In production agent deployments the harness now accounts for the overwhelming majority of source lines, with the model contributing only a small fraction. This structural inversion has a direct security consequence: the harness, not the model, is the dominant attack surface.

Agent harnesses expose several classes of latent vulnerabilities. *Configuration-as-code* attacks exploit declarative configuration that the harness loads and executes without sanitization (CVE-2026-21852 on Claude Code's `.claude/settings.json`; CVE-2026-21858 on n8n automation workflows). *Serialization-injection* attacks exploit the harness's serialization/deserialization of tool responses and agent state, enabling arbitrary class instantiation in trusted namespaces (CVE-2025-68664 on LangChain Core, CVSS 9.3). *Hook-bypass* attacks exploit incorrectly wired security hook points in the agent runtime — as with OpenClaw issues #5513 and #5943, where `before_tool_call` is documented as available but never invoked. *CSRF and session* attacks exploit the harness's web

surface (CVE-2025-34291 on LangFlow, actively exploited in the wild as of January 2026 per CrowdSec [7]).

These vulnerability classes correspond directly to Kavach's minister structure. EXECUTOR detects code-execution-class attacks (CVE-2025-59536, CVE-2025-34291, CVE-2025-68664). VAULT detects credential-theft-class attacks (CVE-2026-21852, CVE-2025-68664). CHANNEL detects exfiltration-class attacks (CVE-2026-21852 via URL redirection, CVE-2026-21858). NAVIGATOR detects trajectory-manipulation attacks that span multiple tool calls. Section 3 gives the MITRE ATT&CK and OWASP Agentic 2026 taxonomic grounding for each minister.

2.2 Existing guardrails and their enforcement points

The guardrail literature can be organized along two axes: (1) *where* in the agent lifecycle the guardrail enforces, and (2) *what mechanism* it uses to detect violations.

Input-time guardrails. operate on the user's message before the agent processes it. PromptGuard 2 [5] (DeBERTa fine-tune, 19.3 ms on A100, 97.5% recall at 1% FPR on English jailbreaks) and Azure Prompt Shields (Microsoft) are canonical examples. These guardrails cannot detect attacks that arrive via tool outputs, which are the dominant attack vector in prompt-injection benchmarks [22].

Plan-time / chain-of-thought guardrails. operate on the agent's reasoning trace before it commits to an action. AlignmentCheck [5] uses Llama 4 Maverick or Llama 3.3 70B to audit the chain-of-thought, reducing AgentDojo ASR from 0.18 to 0.0289 (an 84% reduction) standalone. DRIFT [15] chains a Secure Planner, Dynamic Validator, and Injection Isolator across the plan representation. These guardrails are powerful but expensive (full LLM inference) and cannot inspect the *resolved arguments* of a tool call, only their declarative intent.

Tool-call-time guardrails. operate at the tool-call boundary. This is Kavach's enforcement point. LlamaFirewall's CodeShield [5] applies Semgrep static analysis to code-returning tool outputs, achieving 96% precision and 79% recall on CyberSecEval3 — but only for code generation, not for general tool calls. ShieldAgent [4] retrieves probabilistic rule circuits for each action trajectory. Task Shield [12] evaluates whether each instruction advances the user's task. None of these systems uses a curated semantic corpus of attack-pattern descriptions at the tool-call level.

Information-flow guardrails. tag data with provenance and capability labels, then enforce policies statically. CaMeL [8] (dual-LLM, custom interpreter, 77% task completion with provable security on AgentDojo), FIDES [6] (multi-agent IFC with confidentiality and integrity labels), and RTBAS [24] (attention-based saliency screener that prevents 100% of policy-violating attacks with under 2% utility loss on AgentDojo) belong to this family. Their strength is provability; their limitation is that they cannot detect attacks whose data provenance is correctly labeled but whose argument content is semantically malicious.

Output-time guardrails. operate after the tool has executed, on the agent's reply. NeMo Guardrails (NVIDIA), Conseca [19], and the post-hoc monitor that Kavach replaces belong here. These are useful for forensics and audit but cannot prevent tool execution.

Drift-detection guardrails. measure whether the agent's trajectory diverges from the user's stated intent across turns. TaskTracker [1] uses activation deltas in white-box models. DeepContext [3] uses a turn-level RNN (F1 0.84, sub-20 ms). Trajectory Guard [2] uses a Siamese Recurrent Autoencoder (AAAI 2026 Trustworthy Agentic AI Workshop). Kavach's NAVIGATOR minister belongs to this family at the per-call level, comparing a proposed consequential action against the session's pinned user intent; we report both a completed vocabulary-agnostic taxonomy layer and an incomplete authorization mechanism for this minister in §6.

Across these categories, the tool-call boundary is the only enforcement point at which the *resolved runtime arguments* of an action are available for inspection before that action executes. Input-time and plan-time guards see only declared intent — the user message or the agent's reasoning trace — not the concrete arguments the call will run with; output-time guards observe the action only after it has executed, too late to prevent it; and information-flow guards enforce data provenance rather than the semantics of the argument content. This is Kavach's enforcement point, and it is structurally unreachable by guards in the other categories: they fire at boundaries where the resolved-argument view does not yet, or no longer, exists.

Table 1 (§5) places Kavach in context on five axes: enforcement boundary, mechanism, corpus extensibility, verdict combination, and reported AgentDojo / InjecAgent performance.

2.3 Benchmarks and evaluation methodology

The three most widely used agent-security benchmarks are:

InjecAgent [22] — 1,054 prompt-injection test cases from UIUC (2024), structured as 17 user tools × 62 attacker tools, split into direct-harm (30 tools) and data-stealing (32 tools) settings. Each case provides a benign user instruction, a tool the agent would invoke, a malicious payload in the tool's output, and the expected attacker action. The standard reported metric is Attack Success Rate (ASR) at the harness level. Kavach runs the 544-case data-stealing (DS) subset cold, and the direct-harm subset separately (§4.7).

AgentDojo [9] — 97 user tasks and 629 security test cases across four environments: workspace, Slack, banking, and travel. The standard metrics are ASR (lower is better) and utility score (higher is better for benign tasks). We evaluate against AgentDojo's structurally applicable ground-truth attack cases (§4.5) and, at scale, against all 629 real recorded GPT-4o attack trajectories AgentDojo has published (§4.6).

A methodological note: reporting attack recall without the accompanying benign false-positive rate conflates detector quality with detector aggressiveness. We therefore report benign false-positive counts jointly with recall for every detection number in §4, following the cost-with-detection discipline recommended by Kapoor et al. [14], rather than treating a fixed false-positive ceiling as a gate any configuration must pass before its recall is reported.

2.4 The OWASP and MITRE taxonomic grounding

The two most widely cited threat taxonomies for AI agents as of 2026 are:

MITRE ATT&CK for Enterprise — the standard technique taxonomy, now extended to AI systems via MITRE ATLAS. Kavach's ministers draw patterns from T1059 (Command and Scripting Interpreter), T1546 (Event Triggered Execution), T1552 (Unsecured Credentials), T1539 (Steal Web Session Cookie), T1041 (Exfiltration Over C2 Channel), T1567 (Exfiltration Over Web Service), T1083 (File and Directory Discovery), and others. ShieldNet [21] independently derives 25+ attack types from MITRE ATT&CK for their MCP supply-chain benchmark.

OWASP Top 10 for Agentic Applications 2026 — released December 9, 2025. Ten categories: ASI01 Agent Goal Hijack, ASI02 Tool Misuse & Exploitation, ASI03 Identity & Privilege Abuse, ASI04 Agentic Supply Chain Vulnerabilities, ASI05 Unexpected Code Execution (RCE), ASI06 Memory & Context Poisoning, ASI07 Insecure Inter-Agent Communication, ASI08 Cascading Failures, ASI09 Human-Agent Trust Exploitation, ASI10 Rogue Agents. Kavach's minister mapping: EXECUTOR ↔ ASI05 (Unexpected Code Execution), ASI04 (Agentic Supply Chain Vulnerabilities); VAULT ↔ ASI03 (Identity & Privilege Abuse), ASI09 (Human-Agent Trust Exploitation); CHANNEL ↔ ASI07 (Insecure Inter-Agent Communication), ASI08 (Cascading Failures); NAVIGATOR ↔ ASI01 (Agent Goal Hijack), ASI10 (Rogue Agents).

Using both taxonomies as the authoring source for the attack-pattern corpus (§3.2) ensures that the corpus is grounded in the published threat model of the field rather than in the specific attacks the team happened to have seen.

3 Kavach Architecture

3.1 Overview

Kavach is a runtime guardrail that interposes between an LLM agent's tool dispatch and the tool's actual execution. Every resolved tool call is evaluated by a parliament of four domain-specialized ministers — VAULT (credential and identity abuse), EXECUTOR (dangerous code execution), CHANNEL (data exfiltration and provenance), and NAVIGATOR (unauthorized or drifted agent action) — before a deterministic Speaker combines their verdicts by pure veto (any minister's Block suffices, with no score averaging) into Block, Escalate, or Allow. The system exposes a single HTTP endpoint, `POST /hook/parliament`, called synchronously at the pre-execution hook of the host runtime; a verdict other than Allow prevents the call from executing.

3.2 Deterministic-primary detection: why, and for which ministers

Kavach's ministers were originally cosine-primary: each proposed action was embedded and scored against a curated corpus of attack-pattern descriptions, with a per-minister confidence threshold deciding the verdict. Four independent lines of evidence, developed over the course of this project and detailed as a negative-results narrative below, converged on replacing this design for three of the four ministers.

Evidence 1: a lexical-inflation bug pointed at the fix, not just the bug. An early false-positive investigation found that BM25/IDF term weighting, mixed into the retrieval score to sharpen recall, over-weighted rare generic tokens (`url`, `user`) in the small corpus because L3 code-syntax text had leaked into the lexical index, false-blocking benign calls (`get_webpage`, `read_inbox`). Three fixes were evaluated: a SPLADE sparse-retrieval reranker (rejected — it improves recall, not precision, and adds ∼20 ms to a 78 ms budget); a cross-encoder reranker (rejected — replaces the entire calibrated scoring formula and adds 50–100 ms); and a regex/exact-match layer on L3 syntax tokens specifically, which shipped. This shipped fix is the direct conceptual precursor to full deterministic-primary detection: it worked because the false positives were *structural* (a syntax token, not a semantic concept), and a deterministic match on structure resolved them without touching the semantic layer at all.

Evidence 2: a targeted dense fine-tune traded one failure mode for a worse one. We fine-tuned a low-rank adapter on the frozen BGE encoder (0.537% of parameters, 3 epochs, ∼91 minutes on CPU) to separate attack from benign embeddings directly. It worked precisely as intended on its training distribution: a 7× separation gain, and it fixed the `get_webpage`/`read_inbox` false positives outright (71%→0% on real traffic). But real attack recall on 30 genuine InjecAgent attacks regressed from 63.3% to 26.7%. The root cause was corpus composition, not the fine-tuning method: the training positives were almost entirely LOLBIN-shaped (shell one-liners, living-off-the-land binaries), so the adapter learned to separate *that* shape sharply and, in doing so, pulled genuinely different attack shapes — unauthorized bank transfers, access-grant abuse, both look nothing like `curl | bash` in embedding space — away from the decision boundary it had sharpened. The adapter was correctly not merged.

Evidence 3: the corpus gap this exposed was general, not a training artifact. Adding 62 real business-logic attacks (transfers, access grants) as positive examples revealed they do not match *anything* already in the corpus, technical-surface-only as authored: mean cosine similarity 0.565, far below any operating threshold. This was not a fine-tuning failure; it was evidence that a purely technical attack-pattern corpus has a structural blind spot for attacks whose surface form is an

ordinary-looking business action with a malicious argument, and it directly motivated the next test.

Evidence 4: a proposed fifth minister was formally falsified before being built. We considered STEWARD, a proposed business-logic-specific minister to close the Evidence-3 gap by the same cosine-retrieval mechanism the other ministers used. Before implementing it, we tested whether cosine similarity could discriminate this attack class at the tool-call level at all, using paired legitimate/malicious AgentDojo calls that share a tool name and differ only in argument content (e.g. a legitimate transfer to a known payee versus an attacker-substituted recipient). The similarity deltas between paired calls were near zero (0.003–0.024). Cosine similarity over an action's text representation cannot separate these cases, because the attack signal lives entirely in *which* argument value is present, not in the action's overall semantic shape. STEWARD was not built. This result generalizes beyond the one minister it was proposed for: it is direct evidence that argument-*value* provenance — where did this specific value come from — is a different kind of question than action-level semantic similarity, and no amount of corpus expansion or fine-tuning closes that gap for a mechanism that only ever looks at the action as a whole.

The decision. Taken together, Evidence 1–4 point at the same underlying distinction: attacks whose defining signal is *structural* (a credential-path shape, a network-fetch-to-disk pattern, a specific argument value's provenance) are caught more reliably, more cheaply, and more robustly by deterministic parsing than by any refinement of cosine similarity over an action's semantic embedding — while attacks whose defining signal is genuinely about intent or session-level authorization may still need a semantic or session-state mechanism. We therefore moved VAULT, EXECUTOR, and CHANNEL to deterministic-primary detection, keeping cosine similarity only as an escalate-only triage layer on the deterministic residual (§3.5). NAVIGATOR, whose job is closer to the genuinely intent-shaped question, remains cosine-primary; a direct deterministic swap for NAVIGATOR was attempted and abandoned at its own go/no-go gate (zero net discrimination gain over its cosine baseline), and a second attempt at a deterministic authorization layer is reported as a negative result in §6.

3.3 VAULT and EXECUTOR: structural parsing

VAULT and EXECUTOR score each proposed tool call's arguments with deterministic prefilters (`parliament/prefilters.py`) built on a behavioral-primitive taxonomy: VAULT's primitives (V1–V6) cover credential paths, dotenv files, named-service secret environment variables, and credential-shaped strings; EXECUTOR's primitives (E1–E7) cover network-fetch-to-disk, reverse-shell shapes, and persistence-mechanism writes. Two extraction layers back these primitives, chosen per shell dialect by a latency/correctness trade-off measured directly rather than assumed:

Real AST parsing where it is both correct and fast. For `sh`/`bash` dialects, arguments are parsed with `bashlex` into a real abstract syntax tree, and primitive matching walks the tree rather than the raw text — this closes shape-based false negatives a tokenizer cannot (e.g. heredoc body content, which a naive tokenizer sees as plain text but which bashlex correctly scopes outside the command being executed) at ∼0.5 ms per call, comfortably inside the latency budget. PowerShell has an equivalently real AST available (walking both `CommandAst` and `InvokeMemberExpressionAst` nodes, the latter needed for `.NET` method-call fetches such as `.DownloadFile()`), but invoking it requires a `powershell.exe` subprocess, measured directly at ∼1.4 s per call — 18× over the latency budget. We therefore wire the real AST into the live path only for `sh`/`bash`; PowerShell and `command_prompt` (confirmed via direct test that `bashlex` raises a parse error on real `cmd.exe` batch syntax, which has no equivalently fast compatible parser) remain on the shell-quote-aware structural tokenizer below, with the real PowerShell AST kept available as an opt-in offline/audit tool rather than a live dependency.

A structural tokenizer for the remaining dialects. For dialects where a real grammar parser is unavailable or too slow, a lightweight shell-quote-aware tokenizer extracts URL-, path-, and credential-shaped tokens directly, with a two-tier credential test (a strong extension/path match, and a broader path-shaped fallback) rather than a single regex.

On the full 519-case AtomicRedTeam/GTFOBins reference corpus, deterministic matching alone recovers VAULT 34.2% (40/117) and EXECUTOR 34.5% (70/203) of cases; the remaining residual is handled by the cosine-triage layer below, reaching a full-pipeline VAULT 74.4% (87/117) and EXECUTOR 90.1% (183/203). Of VAULT's 74.4% (87/117), 42 points are hard-Block from the deterministic layer and 45 points are Escalate-only from cosine triage; of EXECUTOR's 90.1% (183/203), 70 points are hard-Block and 113 points are Escalate-only. This produces zero new hard-block false positives across both benign populations we measure (793 general-purpose BFCL calls, 55 agent-shaped benign calls). Escalate rates on the two benign populations are, respectively: VAULT 5.5%/13.4%, EXECUTOR 0%/14.9%, CHANNEL 40.0%/4.5%, NAVIGATOR 21.8%/10.8% (agent-shaped/BFCL). A held-out split of the same population (even/odd case IDs) shows the full-pipeline rate is stable across the split — VAULT 71.2%/77.6%, EXECUTOR 89.2%/91.1% — rather than an artifact of a lucky partition.

3.4 CHANNEL: session-level taint and value provenance

CHANNEL is inherently a multi-call, session-level detector: the attack shape it targets is a sensitive-data *read* followed by an external *send* in the same session, which no single-call regex can express. It maintains two independent pieces of per-session state.

Taint. A call to a known source tool (a domain-specific sensitive-data read — account details, health records, credentials, access history) marks the session tainted. A subsequent call to a known destination tool (external communication: email, Slack messaging, a bank transfer) is BLOCKed if the session is tainted and the destination is not allow-listed; the allow-list is empty by default, since no corpus evidence supports a "trusted" external recipient distinct from an attacker-controlled one. Source and destination are matched by tool name against curated lists, extended during this work with a naming-convention-agnostic tokenizer (so `sendMoney`, `send_money`, and `SendMoney` all match the same primitive) and a behavior-general fallback that checks destination-shaped argument values (an email, IBAN, or URL pattern) for any tool *not* already in the named destination-tool list, so a genuinely novel exfiltration tool is not silently invisible to taint.

Provenance. Independent of taint, every destination-tool call's argument values are classified by where they came from this session: USER_SUPPLIED (present in the user's own original instruction), SELF (the session's own verified identity), OUTPUT_DERIVED (first seen in the output of an earlier tool call this session), or NOVEL (neither). A destination value classified OUTPUT_DERIVED or NOVEL on a consequential call returns ESCALATE, deliberately not BLOCK: an escalation-only verdict here is a considered choice, not an oversight, made specifically because of genuine deployment ambiguity a hard block would get wrong in practice — paying a bill by reading its own file for the recipient IBAN, or replying to the sender of an email that was just read, are both real, common, and legitimate same-session read-then-send chains that must not be treated the same as an attacker-substituted destination.

CHANNEL's locked reference numbers: 5/22 real benign multi-step sessions correctly escalate (0 false hard-blocks) on this mechanism; 5/5 AgentDojo attack cases structurally applicable to CHANNEL's read-then-send shape escalate correctly, confirmed against both the benchmark's minimal `ground_truth()` call sequence (AgentDojo's own API for the expected attack steps) and, separately, against AgentDojo's own full recorded model trajectories for the same cases (§4).

3.5 Cosine-ESCALATE-only triage

For VAULT and EXECUTOR, cosine similarity against the original attack-pattern corpus is retained, but only as a triage layer over the deterministic *residual* — cases the structural parsers did not match — and its verdict is hardcoded to ESCALATE: it can never BLOCK. This is a deliberate narrowing of cosine's role from primary decision-maker to a lower-confidence second opinion, motivated directly by Evidence 4 above: a semantic signal that cannot reliably discriminate should not be trusted with an irreversible verdict, but is still useful as a flag for human review on cases a deterministic rule did not anticipate.

3.6 An LLM tiebreaker for narrative-phrasing attacks (built, not deployed)

VAULT and EXECUTOR's deterministic rules match literal syntax; an attack described in prose rather than code ("append an entry to the SSH authorized-keys file for persistence" rather than the literal path) can evade every rule that only recognizes concrete command shapes. We built an escalation tier for this residual that keeps the verdict-making step model-free even though it consults a model: a cheap deterministic pre-flag decides whether a call is worth escalating at all, a small local model (`qwen2.5:7b`) only *extracts three narrow factual flags* from flagged calls rather than rendering a verdict directly, and a separate deterministic rule converts those flags into BLOCK/ESCALATE/ALLOW (failing open to ALLOW on model timeout or error). Two simpler designs — raw entity-proximity matching and asking the model to judge malicious-or-benign directly — were tried and rejected first, for benign false positives and missed attacks respectively. The tiebreaker scored 16/17 (94.1%) in pilot, and 4/5 on an independent generalization check using phrasings never used to write its fix (§4.4); it is real, working code held back from live dispatch specifically because its own validation found a limit before we judged it decided.

3.7 Provenance ledger

Every verdict is logged to an append-only SQLite ledger with a SHA-256 hash chain: each row's `entry_hash` is computed over the previous row's hash plus a canonical serialization of that row's content-bearing fields, so editing, deleting, or reordering any historical row breaks every subsequent hash, detectable in $O(n)$ by `/ledger/verify`. The chain is tamper-*evident*, not tamper-*proof*: an attacker with database write access can recompute the whole chain, and defeating that requires an external append-only anchor, left as future work. The hashing scheme is deliberately schema-evolution-tolerant — a field added after the ledger was already in production only enters the hash for rows written after the field existed, so old rows continue to verify under the same code that verifies new ones. We rely on this property directly in practice: during this work we found and fixed a case where the *verification* query, not the ledger itself, omitted a newer column from its recomputation, producing a false tamper report at the exact row where that column was first populated; correcting the query (not the stored data) restored a clean `intact: true` verification across all rows. The episode is itself evidence the tamper-evidence mechanism is exercised and audited in practice, not merely specified.

3.8 Design decisions and trade-offs

Deterministic-primary is a per-minister decision, not a blanket policy. We did not move all four ministers to deterministic-primary detection uniformly; NAVIGATOR's question — was this specific action authorized by what the user actually asked for — does not reduce to a structural shape the way a credential path or a network-fetch-to-disk pattern does. Treating

this as a per-minister empirical question, rather than assuming one mechanism generalizes to all four, is itself a finding we carry into §6's report of the NAVIGATOR authorization attempts.

Escalate, not block, where the deployment ambiguity is genuine. CHANNEL's provenance mechanism escalates rather than blocks specifically because we found real, common, benign same-session read-then-send patterns in actual trajectory data that a hard block would incorrectly stop. Where a verdict boundary has a real false-positive cost we can point to concretely, we chose the reversible verdict.

Escalate has a real operational cost, and we report it as such. An Escalate verdict is a request for human review, not a prevented action — the call still reaches the user or operator queue, and someone has to look at it. This has a real cost on legitimate traffic: on the 55-case agent-shaped benign population, 37/55 (67.3%) trigger at least one Escalate somewhere in the pipeline — a majority of ordinary calls, not a tail case. VAULT and EXECUTOR's full-pipeline detection numbers (§3.3) already report the Block/Escalate split for exactly this reason: VAULT's 74.4% is 42 points hard-Block from the deterministic layer and 45 points Escalate-only from cosine triage; EXECUTOR's 90.1% is 70 points hard-Block and 113 points Escalate-only. A reader who reads either number as "prevented" without checking this split will overstate what the deployed system prevents versus flags for review.

Synchronous evaluation on the hot path. Kavach evaluates synchronously in the pre-execution hook. Asynchronous evaluation would reduce latency but allow a race between the verdict and execution; we accept the latency cost to maintain the guarantee that no tool executes before a verdict is issued.

A pre-rearchitecture session monitor remains active alongside the four ministers. TRAJECTORY predates this project's rearchitecture (§3.2) and was not moved to deterministic-primary detection; it combines five signals — accumulated per-call confidence, drift from a seeded session intent, pairwise action coherence, denial-echo detection, and cross-minister escalation — into a single per-session risk score, reusing each call's already-computed embedding. Its escalation signal shares the same underlying mechanism as the standalone intent-drift detector reported as a negative result in §6. TRAJECTORY is not a first-line detector: across every locked evaluation, zero cases counted toward a reported detection number were first identified by TRAJECTORY — a minister always fires first. It instead re-confirms or escalates a subset of already-detected cases at the session level: 148 of the 629 real GPT-4o trajectories in §4.6 (none affecting the reported 467/629), and, notably, 3 of the 22 real benign AgentDojo sessions in §4.5 — a hard Block, jointly with NAVIGATOR, on benign traffic our own escalate-only CHANNEL mechanism does not produce. We report this plainly: it is a real false-positive-producing interaction on a live component we did not subject to the same validation rigor as VAULT, EXECUTOR, and CHANNEL, and it is not a contribution of this paper.

4 Evaluation

We evaluate Kavach on three independent axes: deterministic-primary detection on Kavach-PB, a composite benchmark we built specifically because the two standard benchmarks below almost entirely miss VAULT and EXECUTOR (§4.2–4.3), CHANNEL's session-level provenance mechanism against real multi-step attack and benign trajectories from AgentDojo (§4.5), and a large-scale real-trajectory validation against AgentDojo's own recorded model transcripts, not a benchmark replay of minimal ground-truth call sequences (§4.6). Throughout, we report benign false-positive counts jointly with attack recall, and we distinguish measurements taken via in-process scoring from measurements taken via the live HTTP dispatch path, since these two paths previously diverged in ways that mattered (§4.1).

4.1 An audit discrepancy, and how it was resolved

Early in this work, three different measurements of VAULT/EXECUTOR detection rate on the same reference corpus disagreed sharply: 34.2%/34.5% (deterministic layer alone, in-process), 72.6%/89.9% (full pipeline, in-process), and 14.9%/19.2% (an earlier live-HTTP measurement). We traced this to two independent causes rather than accepting any one number at face value: the live-HTTP figure was corrupted by a duplicate server process answering a fraction of requests, and, separately, the metric itself differed — one measurement counted Block-or-Escalate as a hit while another counted only the deterministic layer's contribution. Once both were corrected, the live-HTTP and in-process full-pipeline numbers agreed. The episode shows how easily a benchmark harness's own bugs can masquerade as detector performance — a risk equally present in any comparably complex evaluation pipeline, ours included.

4.2 Kavach-PB: a composite benchmark for all four ministers

InjecAgent and AgentDojo, the two standard agent-security benchmarks we rely on elsewhere in this section, almost exclusively stress CHANNEL: InjecAgent's attack shape is a read-then-send taint chain, and AgentDojo's is a provenance-mismatched destination value on a consequential call. Neither meaningfully exercises VAULT (credential and secret handling) or EXECUTOR (dangerous or injected code execution) at all. We found this gap live, while validating VAULT/EXECUTOR's deterministic-primary rearchitecture (§3.2) and discovering we had no real benchmark to validate it against — not an anticipated design requirement.

Rather than author new synthetic attacks to close this gap, we built **Kavach-PB** (Kavach Parliament Benchmark): a unified, composite adversarial evaluation suite that reuses already-real, externally-validated attack data under one runner, one wire format, and one scoring convention across all

four ministers. This choice is deliberate and methodological, not incidental: a self-authored attack set risks unconsciously being shaped to be catchable by the very system it evaluates, since the same person or process who understands the detector's mechanism is the one writing the attacks. Reusing attack corpora authored independently, by real security researchers, for a purpose unrelated to Kavach, avoids this risk structurally rather than by discipline alone.

What is reused, what is new. The underlying attack and benign data is real and reused, not invented: attack cases are drawn from AtomicRedTeam (technique-tagged adversary emulation commands) and GTFOBins (Unix binary living-off-the-land technique commands), and benign calls are drawn from the Berkeley Function-Calling Leaderboard (BFCL, general-purpose) plus a 55-case agent-shaped benign set. What is novel is the unification itself: a single harness dispatching every case through the same `/hook/parliament` path regardless of source corpus, a per-minister scoring methodology (below), and a two-population benign false-positive convention — none of which existed before this evaluation, all of which are needed to make VAULT/EXECUTOR measurable at all under real, externally sourced attack data.

Scoring convention. Each attack case carries a `target_patterns` list (e.g. `["VAULT:VAULT-060"]`) naming which minister(s) the case is expected to trigger; we report both an overall detected rate (any minister returns BLOCK or ESCALATE, the minister-agnostic bottom line) and a per-minister hit rate (did the specifically targeted minister itself fire, checked directly in the response's per-minister field rather than only via which minister the speaker's verdict is attributed to) — a case can be detected overall while its targeted minister misses, and we surface that divergence as a real coverage finding rather than absorbing it into the aggregate. On the benign side, BLOCK is a hard false positive and ESCALATE is reported separately, never conflated with either BLOCK or a clean ALLOW — the same convention CHANNEL's own provenance mechanism uses (§3.4). We report two benign populations separately rather than pooled: BFCL (793 cases, real but general-purpose/utility calls, not agent-shaped — a clean result here is real but weaker evidence, a domain mismatch we state rather than obscure) and the 55-case agent-shaped benign set (the stronger evidence).

4.3 Kavach-PB results: VAULT and EXECUTOR

On Kavach-PB's 519-case AtomicRedTeam/GTFOBins reference corpus (117 VAULT-targeted, 203 EXECUTOR-targeted cases; the corpus's `target_patterns` field is unmodified since its original authoring commit, confirmed via `git diff`, so it is independent ground truth rather than tuned after the fact), the deterministic layer alone recovers VAULT 34.2% (40/117) and EXECUTOR 34.5% (70/203); adding the cosine-escalate-only triage layer over the deterministic residual (§3.5) raises full-pipeline detection to VAULT 74.4% (87/117) and EXECUTOR 90.1% (183/203), with zero new hard-block false positives on either benign population (793 BFCL, 55 agent-shaped). A held-out even/odd split of the same population confirms the full-pipeline rate is stable rather than an artifact of the specific partition: VAULT 71.2% (42/59) on the even half versus 77.6% (45/58) on the odd half; EXECUTOR 89.2% (91/102) versus 91.1% (92/101). We report both halves explicitly rather than only their average, consistent with our standing discipline of never characterizing a split as "balanced" or "generalizing" from an aggregate number alone.

This split addresses partition luck, not rule-authoring circularity: the deterministic rules were authored by inspecting misses across the full corpus, so both halves were visible during rule development. The generalization audit (§6.1) is our attempt to measure past this limit, and we note its own limit in turn: its scenarios are author-written, testing vocabulary novelty rather than an independently sourced attack distribution.

4.4 LLM tiebreaker: validated, deliberately not deployed

The tiebreaker (§3.6) scored 16/17 (94.1%) in pilot. Re-validated against all 144 real benign-set calls, it produced one new false positive (a `non_standard_source` misread of legitimate internally-authored, locally-stored content); a prompt clarification fixed this case with zero new false positives on re-test. A genuine generalization check — five phrasings never used to write the fix, plus three negative controls — then scored 4/5, with one confirmed remaining miss (an internal file described as run "from the ops share," a network-share phrasing the fix does not cover). We stopped at this result rather than re-running it against our full narrative-phrasing test set: a mechanism with one already-known, characterized generalization gap should not be scored as if it had none, and reporting a headline recall number here would overstate what the 4/5 result actually shows. The tiebreaker is real, working code, held back from live dispatch specifically because its own validation found a limit before we judged it decided.

4.5 CHANNEL: session-level provenance

CHANNEL's provenance mechanism (§3.4) is evaluated on two real populations: 22 real multi-step benign sessions and the AgentDojo attack cases whose two-call read-then-send shape is structurally applicable to CHANNEL's mechanism. On the benign population, 5/22 sessions correctly escalate via CHANNEL, with 0 hard-block false positives from CHANNEL itself; these are exactly the sessions in which a legitimate destination value (an IBAN, most commonly) was read from a file or transaction history earlier in the same session rather than typed by the user, which is precisely the ambiguity that motivated an escalate, not block, verdict for this mechanism (§3.4). Separately, 3/22 sessions in this same population are hard-blocked by an unrelated interaction between TRAJECTORY and NAVIGATOR, never blended into CHANNEL's 5/22 figure and reported in full in §3.8. On the attack side, 5/5 structurally applicable AgentDojo cases correctly escalate,

and we verified this twice: once against the benchmark's minimal `ground_truth()` call sequence, and again, independently, against AgentDojo's own full recorded model trajectories for the same five cases — real, longer sessions with additional benign calls interleaved, real tool outputs, and the actual sequence a deployed model produced rather than the benchmark's answer key. This second check caught a bug in our own verification script (it tracked only the final step's verdict, so a benign call after the real attack step silently overwrote an earlier correct escalation) before we could report the number as final; the corrected script confirms genuine 5/5 on the real trajectories.

4.6 Large-scale real-trajectory validation

The measurements above use either a synthetic reference corpus or a small, hand-verified set of AgentDojo cases. To validate detection at scale on real model behavior, we replayed all 629 real recorded attack trajectories for GPT-4o (`gpt-4o-2024-05-13`) across AgentDojo's four suites' `important_instructions` attack family, sourced directly from AgentDojo's own published `runs/` data — real conversation transcripts with real tool calls and real tool outputs, not a benchmark replay of minimal ground-truth sequences.

Headline result. Kavach fires (BLOCK or ESCALATE, any minister, at any point in the session) on 467/629 (74.2%) of these real trajectories: banking 119/144 (82.6%), slack 105/105 (100%), travel 63/140 (45.0%), workspace 180/240 (75.0%).

Diagnosing the apparent misses. Of the 162 trajectories on which Kavach did not fire, we traced every case to its actual tool-call content rather than reporting a bare miss rate. AgentDojo's own `security` field, which we cross-reference, marks a trajectory as a successful attack whenever the environment state changed at all relative to the pre-trajectory state and does not exactly match the injected goal — which conflates "the injected attack succeeded" with "the model made no consequential call related to the injection at all." 131 of the 162 misses fall in the latter category: the model never invoked any consequential tool in connection with the injected instruction, so there was no unauthorized action for any minister to catch. We verified this directly by extracting every tool call from each miss's recorded transcript and checking it against a fixed consequential-tool list; a trajectory whose only calls are read-only lookups (hotel ratings, restaurant listings, calendar reads) is not a detection gap, regardless of what AgentDojo's own field reports.

The remaining 19 real misses, categorized by mechanism. The other 19 trajectories do contain a genuine unauthorized consequential call that Kavach did not catch, and we categorize each by whether the underlying attack has a destination *value* to trace (CHANNEL's domain) or is purely an unauthorized *action* with no data flow at all (a question closer to NAVIGATOR's domain, §6). Five cases (`travel`, all `reserve_hotel`) name an attacker-substituted external party — a real destination value, of the same shape as `send_money`'s recipient IBAN — but the tool was never added to CHANNEL's destination-tool list, a scope gap rather than a mechanism failure. Fourteen cases (`banking`: 11 cases, `update_scheduled_transaction` and `update_user_info`; `workspace`: 3 cases, `delete_file` and `append_to_file`) mutate existing account state or same-account data with no new destination value introduced at all — there is no value for any provenance mechanism, however general, to trace, because the question these attacks pose is "should this action have happened," not "where did this value come from." We attempted to close the five `reserve_hotel` cases by adding the tool to CHANNEL's destination-tool list, and found this introduces a new false positive: CHANNEL's destination-value extractor recognizes only email/IBAN/phone/URL shapes, so a proper-noun business name such as a hotel is never recognized as "present in the user's own instruction" even when it literally is, causing a legitimate reservation the user explicitly named to be misclassified NOVEL and incorrectly escalate. We reverted this change rather than ship a fix with a known new false-positive class, and report the gap as a documented scope limitation (§6) rather than a closed fix.

Scope of this validation. This validation covers one model (`gpt-4o-2024-05-13`) and one attack family (`important_instructions`), because these are the complete recorded trajectories AgentDojo publishes; whether detection transfers to other models' tool-calling styles or to subtler injection framings is untested. We note the trajectories are real model behavior rather than synthetic replay, a deliberate trade of breadth for realism.

4.7 InjecAgent

InjecAgent's full corpus is 1,054 cases: 510 direct-harm (DH) and 544 data-stealing (DS), each the cross product of InjecAgent's 17 user tasks with its 30 DH / 32 DS attacker instructions. We dispatch DS cases as the two-step sequence they describe — a source read, then a destination send — specifically so CHANNEL's session-scoped taint tracker can observe the read-then-send pattern; DH cases remain single-step.

On DS, Kavach achieves **544/544 (100%) strict (hard-Block) recall**, entirely via CHANNEL: every one of the 544 cases is decided by CHANNEL's taint mechanism (`CHANNEL-TAINT :read-then-send`), with VAULT and EXECUTOR returning ALLOW on all 544 — this result exercises CHANNEL's multi-step provenance/taint mechanism directly, not VAULT/EXECUTOR's deterministic layer.

DH tells a different, weaker story: strict recall is **0/510 (0%)** — no DH case reaches hard BLOCK. 221/510 (43.3%) reach the loose (block-or-escalate) level, split across CHANNEL (153), VAULT (34), EXECUTOR (17), and NAVIGATOR (17); the remaining 289/510 (56.7%) are missed entirely (ALLOW). DH is a genuine, unresolved gap in the deterministic layer, not a strength of the current architecture, and is not folded into DS's headline number.

On the paired benign side (1,054 user-instruction prompts, one per attack case), hard-block FPR is **0%** — no benign prompt is ever hard-blocked — but the soft-escalate rate is

35.2% (371/1,054), again led by CHANNEL (248), with EXECUTOR (62) and NAVIGATOR (61) contributing the rest and VAULT at 0. DS's strong recall and DH's weak recall should both be read against this FPR context, not in isolation.

4.8 Cost

Kavach's latency is bimodal, and we report the split rather than a blended figure. 17 of 159 timed calls complete under 90 ms (p50 46 ms, p95 86 ms); the remaining 142 complete in over 680 ms (p50 733 ms, p95 1437 ms), with no call falling in between. The split does not cleanly correspond to whether a deterministic rule short-circuited the call: 26 of 159 calls were short-circuit-eligible and matched, of which 17 returned in the fast mode and 9 did not, despite an identical decision path. We traced this to the pipeline's ledger-completeness design: every response — short-circuited or not — spawns a background task that runs the skipped COMPASS, router, and NAVIGATOR scoring for audit purposes, competing for the same CPU-bound embedding inference as the next request's own foreground call under sequential dispatch. This is a scheduling artifact of our single-process CPU measurement, not a property of the detection logic itself; a production deployment would either bound or defer this background completion work. On GPU-equipped hardware, we have separately observed the embedding path at approximately 78 ms p50 (RTX 4090, steady state). Verdicts are hardware-independent: replaying 107 reference cases with the encoder pinned to CPU produced zero verdict or fired-minister differences against the GPU-measured baseline. The bashlex AST parse used for `sh/bash` (§3.3) measures ∼0.5 ms per call and is negligible on either path.

5 Related Work

Positioning relative to concurrent runtime work. Microsoft Execution Containers (MXC, announced Build 2026) and ClawGuard [23] both harden the OpenClaw runtime Kavach was originally built against. MXC is an OS-level policy sandbox that reduces blast radius but cannot reason about intent: an agent authorized to read credential files and make HTTP calls can still be manipulated into exfiltrating those credentials via indirect injection while remaining fully within its MXC policy — the argument-content question Kavach answers is orthogonal to a capability sandbox. ClawGuard reports a 0% attack-success rate on AgentDojo under explicit, hand-authored rules; its automatic rule-derivation component is described as future work, so that number reflects hand-authored coverage of the specific benchmark rather than a generalizing detector, a limitation its own authors acknowledge for content-misleading attacks specifically. DeepContext [3] and PSG-Agent [20] are architecturally adjacent — a recurrent session-state tracker and a four-component guardrail respectively — and are positioned in Table 1; our differentiation from both is the same one that runs through this paper: mechanism choice (deterministic vs. cosine) is made per detection domain on direct evidence rather than applied uniformly, and we report where each choice does and does not generalize (§6) rather than only where it succeeds.

Plan-time authorization systems. The closest external precedents for NAVIGATOR's authorization question operate at the plan level rather than the tool-call level. PlanGuard [11] constructs an isolated plan from the user's instructions alone, then gates execution with a hard-constraint stage followed by an LLM-based intent verifier, reporting 100% attack-blocking on InjecAgent. NAVIGATOR's pinned-intent design is analogous to PlanGuard's first stage, and reduces authorization to a deterministic question specifically to keep the enforcement decision model-free — but where PlanGuard reports a working mechanism, we report our hard-constraint layer as an incomplete, three-attempt negative result (§6). We read the comparison as informative in both directions: PlanGuard's LLM-based verifier may be doing real work that our deterministic heuristics did not find a way to replace, and our three characterized failure modes indicate what any deterministic replacement for such a verifier must overcome. IntentGuard [13] performs instruction-following intent analysis with origin tracing and treats user-confirmation-on-alert as a first-class mechanism rather than a fallback; this is close in spirit to CHANNEL's deliberate escalate-not-block verdict on genuine provenance ambiguity (§3.4), though IntentGuard applies it at the plan level and Kavach at the argument-provenance level. Progent [18] enforces a progressive per-tool allowlist policy and reports an AgentDojo ASR reduction from 39.9% to 1.0%; we cite this as an upper reference point for what a mature policy layer can achieve rather than a number Kavach reproduces, since policy synthesis and deterministic argument parsing are not directly comparable mechanisms.

Red-team and scanning tools. garak [10] (NVIDIA) and PyRIT [16] (Microsoft) are offensive test harnesses that probe a model or agent for vulnerabilities pre-deployment rather than enforce at runtime, and LLM-Guard [17] is an input/output scanner suite operating at the prompt boundary rather than the tool-call boundary. These are complementary rather than competing: Kavach is a runtime, tool-call-time enforcement layer that such tools can be used to red-team, and we would welcome exactly that.

Table 1 (Appendix B) situates Kavach against 26 systems on five axes: enforcement boundary, mechanism, corpus extensibility, verdict combination, and reported benchmark numbers. Restricting to the three systems discussed above: PlanGuard reports 100% attack-blocking on InjecAgent with a plan-level LLM verifier NAVIGATOR's authorization layer does not yet have a deterministic equivalent for; ClawGuard reports a 0% AgentDojo ASR under hand-authored rules, the same benchmark-coverage caveat that motivates our own generalization audit (Appendix A); and Progent reports an AgentDojo ASR reduction from 39.9% to 1.0% via a progressive allowlist policy, an upper reference point for policy-layer enforcement rather than a number Kavach's argument-parsing mechanism reproduces. The numbers in Table 1 are drawn from each cited paper's own evaluation setup — benchmark subset, environment version, and threat model all vary

system to system — so the table should be read as a map of enforcement boundaries and mechanisms, not a benchmark leaderboard; no number in it is directly comparable across rows. A full 26-system comparison appears in Appendix B.

6 Limitations

No head-to-head with ClawGuard specifically. An earlier version of this work lacked any AgentDojo evaluation at scale; that general gap is now closed by the 629-real-trajectory validation in §4.6. A narrower gap remains: ClawGuard [23], a specific hand-authored-rule tool-call interceptor, reports a 0% attack-success rate on AgentDojo under its own rule set, and we do not run a controlled head-to-head against ClawGuard's own implementation on a shared harness. ClawGuard's public implementation intercepts live sessions with no dataset-replay harness compatible with our evaluation pipeline, so a direct comparison remains future work; we note that ClawGuard's authors report its automatic rule-derivation component as unevaluated, so its published number reflects hand-authored coverage of that specific benchmark, which any head-to-head would need to account for rather than treat as a like-for-like baseline.

Tamper-evident, not tamper-proof. The hash-chained ledger detects post-hoc edits to any recorded decision or its provenance, but an attacker with write access to the database can recompute the entire chain. Defeating that requires anchoring the chain head to an external append-only medium, which we do not implement. We note, as a point in favor of the mechanism being exercised rather than only specified, that we found and fixed a real bug in the ledger's own *verification* query during this work (§3.7) — the kind of bug that only surfaces when the tamper-evidence mechanism is actually run end to end against real data, not when it is merely reviewed as a design.

Curated corpus and deterministic rule set. Both the cosine-triage corpus and the deterministic prefilters' rule sets are authored against published taxonomies (MITRE ATT&CK/ATLAS, AtomicRedTeam, GTFOBins) and known attack corpora (InjecAgent, AgentDojo). A genuinely novel technique outside both may not be caught; §6.1 characterizes this precisely for each minister rather than leaving it as an untested assumption.

The reserve_hotel gap is representative of a broader, unfixed class. §4.6 reports 19 real trajectories where a consequential attack succeeded and Kavach did not fire, split between a narrow CHANNEL scope gap (five `reserve_hotel` cases, where a destination value exists but the tool was never added to CHANNEL's destination list) and fourteen cases with no destination value to trace at all. We attempted to close the CHANNEL gap and found the natural fix — adding the tool to the destination list — introduces a new false positive, because CHANNEL's value extractor only recognizes email/IBAN/phone/URL shapes and not proper-noun business names; a legitimate reservation the user explicitly named would then misclassify as unauthorized. We reverted this change rather than ship a fix with a known new false-positive class. The fourteen cases with no destination value at all are, structurally, not closeable by any refinement of CHANNEL's mechanism, however general its value extraction becomes — they are exactly the shape of question §6 below reports as unresolved.

Single embedding model. The cosine-triage layer and NAVIGATOR use a single encoder (`bge-base-en-v1.5`), selected for open-source offline reproducibility rather than by a comparative ablation; whether an alternative open-source retrieval encoder would shift the residual detection rate is an open question we leave to future work.

PowerShell's real AST is correct but too slow for the live path. §3.3 reports a measured ∼1.4 s per call for a real PowerShell grammar parse via subprocess, 18× over the latency budget, so PowerShell calls are scored by the structural tokenizer rather than a real AST in the live path. The real parser remains available as an offline/audit tool. A faster in-process PowerShell AST (avoiding the subprocess-spawn cost) is a concrete, scoped next step we did not pursue this cycle.

NAVIGATOR authorization layer: a hard-constraint layer still in progress, after three failed heuristics. NAVIGATOR is the one minister whose deterministic rearchitecture is not yet complete. A direct deterministic swap mirroring VAULT/EXECUTOR/CHANNEL showed zero net discrimination gain over NAVIGATOR's cosine baseline and was abandoned; a second attempt reframes the minister's question as authorization rather than similarity (a pinned, immutable session intent, classifying each tool as observational or consequential, then a hard-constraint layer deciding whether a consequential action is *derivable* from that intent). The first two pieces—pinned-intent capture and the taxonomy—are complete and validated, including under the generalization audit below. The hard-constraint layer itself has failed three independently designed heuristics against a frozen discriminability criterion set before measurement: (1) tool-name/instruction-text overlap, evaded by attacker text sharing surface words with the legitimate request; (2) argument-value overlap alone, which wrongly flagged legitimate values read earlier in the same session; (3) a provenance-aware version of (2), composed read-only with CHANNEL's own session state, which still exceeded the frozen false-positive threshold. We report only completed, validated results as the contribution in this version of the paper; the hard-constraint layer remains open, and its three failure modes are reported precisely because a negative result with a specific mechanism of failure is more useful to future work than a vague one.

A proposed alternative for NAVIGATOR: intent-drift, explored and not validated. Independently of the hard-constraint layer above, we explored a second candidate mechanism for NAVIGATOR — flagging a tool call as drift when its embedding similarity to the session's original user-stated goal falls below a threshold — which reached 95.2% recall at 0% false-positive rate on an initial 100-case dataset (83 attack, 17

benign), but did not generalize: enlarging the benign sample from 17 to 39 with real multi-step AgentDojo sessions raised the false-positive rate to 35.9% at the same threshold, because ordinary mid-session follow-up actions (e.g., checking a balance right after a payment) are semantically distant from the session's *original* goal despite being legitimate, and held-out recall on the enlarged set collapsed to 22–56%. This is consistent with an independent evaluation of the same mechanism on AgentDojo ground-truth, which measured 7.3% recall. We report this as future work, not a contribution: NAVIGATOR's deployed detector remains the cosine-corpus mechanism in §6.1, unchanged.

No adaptive-attacker evaluation. Every attack corpus in this paper is static: no attacker in our evaluation knows Kavach's rules or adapts to them. Deterministic signature matching is historically evadable — encoded payloads, string construction, variable indirection, `sh -c` with assembled commands — and while bashlex's real AST closes a class of these transforms for sh/bash, PowerShell and cmd remain on a tokenizer, and no minister has been stress-tested against an attacker optimizing to evade it. An obfuscation-transform study over the 519-case reference corpus — measuring detection degradation under encoding, splitting, and indirection — is the immediate next step, and until it is done, the detection rates in §4 should be read as rates against non-adaptive attacks.

6.1 Generalization beyond the benchmark vocabulary

Kavach's deterministic rules were, in large part, authored by mining detection misses on the exact benchmarks evaluated against (§4.2), creating a real risk of circular validation. We therefore ran an independent adversarial audit before submission: a fictional agent environment ("AtriumOS") with an entirely invented tool vocabulary sharing no tool name, corpus term, or surface phrasing with InjecAgent, AgentDojo, or Kavach-PB, dispatching 18 attack and 18 matched-benign novel-vocabulary scenarios through the real detection pipeline. The per-minister verdict: EXECUTOR generalizes cleanly (caught all eight novel-vocabulary attacks; one structural false positive was found and fixed pre-submission); VAULT mostly generalizes, with one limitation inherent to the architecture (a literal secret-name allowlist cannot match a genuinely novel secret-variable name, by construction); CHANNEL is split — its escalation path generalizes via a behavior-general destination-value fallback, but its hard-block path does not, since destination status is decided by a benchmark-specific tool-name list; and NAVIGATOR's cosine-similarity mechanism does not currently transfer to novel vocabulary at all, a direct structural consequence of the same lexical gate that gives it low false positives on-benchmark. Full methodology and results in Appendix A.

7 Conclusion

Kavach demonstrates that deterministic parsing over a tool call's resolved arguments outperforms cosine similarity over its semantic embedding for detection problems whose signal is structural, and that this is a per-mechanism empirical question rather than a property of the detection paradigm as a whole: three of Kavach's four ministers moved to deterministic-primary detection on four independent lines of evidence, while the fourth's genuinely intent-shaped question remains open past a validated taxonomy classifier. Validating this pipeline against 629 real recorded attack trajectories, rather than benchmark replay alone, surfaced both real detection strength and a small number of precisely characterized gaps. We take the negative results in this paper — the falsified fifth minister, the abandoned deterministic swap, the three failed authorization heuristics, and the generalization audit's per-mechanism boundaries — to be as much a contribution as the detection numbers: knowing where a runtime monitor's mechanisms do and do not generalize is what makes its positive results trustworthy.

A Generalization Audit Methodology and Full Results

Why we audit this, and how. Kavach's deterministic detection rules were, in large part, authored by mining detection misses on the exact benchmarks we evaluate against: the VAULT and EXECUTOR rule sets were expanded by inspecting which of the 519 AtomicRedTeam/GTFOBins cases failed and writing a rule for each missed technique, and CHANNEL's destination-tool list and provenance patterns were built and expanded against InjecAgent and AgentDojo cases specifically. This is legitimate engineering, but it creates a real risk of circular validation: testing detection against the same data shape it was derived from will always flatter the detector. We therefore ran an independent, adversarial generalization audit before submission. We constructed a fictional agent environment—"AtriumOS," a smart-building/industrial-facility control agent—with an entirely invented tool vocabulary (`FacilityConfigRead`, `Hvac Diagnostic`, `UnlockPerimeterGate`, `RetrieveTenantAccessLog`, and so on) that shares no tool name, corpus term, or surface phrasing with InjecAgent, AgentDojo, or Kavach-PB (§4.2). We then dispatched 18 attack and 18 matched-benign scenarios—covering each minister's intended detection *shape* (credential exfiltration, dangerous execution, data-flow-to-untrusted-destination, unauthorized consequential action) in this novel surface form—through the real detection pipeline, and measured per-minister hit/miss per case. This sits in real tension with our own argument against self-authored attack data (§4.2): the audit's scenarios are author-written, not independently sourced. We consider this acceptable specifically because the audit tests *transfer* of already-validated mechanisms to novel vocabulary, not a headline detection claim—the mechanisms being stressed were built and locked against Kavach-PB, InjecAgent, and AgentDojo before AtriumOS existed, so the risk Kavach-PB's own design guards against (a detector unconsciously shaped to catch what it will be tested on) does not apply to a novel-vocabulary check run against mechanisms already frozen. We report the findings

without softening: the per-mechanism boundaries below are a deliberate product of this pass, and are more useful stated precisely than defended.

EXECUTOR generalizes; one structural false positive was found and fixed. EXECUTOR caught all eight novel-vocabulary execution attacks (`curl | bash`, `os.system`, `eval` of a decoded payload, an `nc` reverse shell, cron-drop-in persistence, a `sudoers` write, an inline-Python reverse shell, and a network-fetch-to-disk) despite novel hosts, tool names, and file paths, because these rules match on structural and technique-level shape rather than on tool identity. This is the strongest generalization result of the four ministers. The audit did surface one genuine defect: the structural network-fetch-to-disk predicate (`is_network_fetch_to_disk`) fired on *any* URL-plus-write-destination shape, including a benign `curl http://host/api/status -o /tmp/status.json` data fetch—re-introducing exactly the false positive the regular-expression layer had already excluded when it deliberately dropped its bare `curl -o` branch. We scoped the structural predicate to require a payload-staging signal (a pipe into an interpreter, or a write target carrying an executable/script/library/archive extension), mirroring the regex layer's own exclusion logic. Post-fix validation on the full 519-case attack corpus and both benign populations: the audit's false positive is now silent; genuine network-fetch-to-disk attacks still fire; EXECUTOR's attack hit count moved by a single case (98 → 97), which on inspection is a CHANNEL-targeted HTTP-exfil scenario on which EXECUTOR had been firing *incidentally* on a benign-shaped data-retrieval command, not a genuine execution attack; and both benign populations—793 general-purpose (BFCL) and 55 agent-shaped—stay at zero hard-block false positives. The parser test suites remain green (27/27 and 37/37).

VAULT mostly generalizes, with one inherent limitation. VAULT caught five of seven novel-vocabulary credential attacks—SSH and `.pem` private-key paths, a hard-coded AWS access-key-id, a credential-file search by extension, and a generic `secrets/` path—all via structural credential-path and secret-format matching that is indifferent to the surrounding tool name. Two misses are worth distinguishing. The first is a narrow surface-form gap: a file named `badgereader.env` carries `.env` as an *extension* rather than as the canonical dotenv *filename*, and the `dotenv-file` rule anchors on the latter. The second is *inherent* to the architecture, not a bug: reading a novel environment-variable secret (`os.environ['ATRIUM_ADMIN_SECRET']`) is not caught because VAULT's named-service environment-key rule is, by construction, a literal allowlist of known secret variable names (`STRIPE_SECRET_KEY`, `ANTHROPIC_API_KEY`, and so on). A genuinely novel secret-variable name cannot match a name-allowlist, regardless of technique—this is a property of literal-name matching, and no expansion of the list closes the general case.

CHANNEL is split: escalation generalizes, hard-block does not. The audit separates CHANNEL's two mechanisms cleanly. The provenance/escalation path *does* generalize: its behavior-general destination-value fallback extracted an attacker email address from a novel send-tool's arguments and returned ESCALATE (classified NOVEL), even though the tool name appears in no benchmark. The taint/hard-block path does *not*: whether a call is a destination is decided by a dictionary lookup against benchmark-specific tool names (`send_email`, `send_money`, `GmailSendEmail`, ...), with no behavior-general fallback, so a novel exfiltration tool (`send_report_email`, `PostToExternalDashboard`) is invisible to the taint mechanism and the read-then-send BLOCK never fires. The guarantee is therefore mechanism-specific: CHANNEL's hard-block guarantee currently holds only within the benchmark's destination-tool vocabulary, while its escalation guarantee is genuinely broader.

NAVIGATOR does not currently generalize as a detector. NAVIGATOR's per-call detector is cosine similarity against an attack-pattern corpus, and the audit shows this signal does not transfer to novel vocabulary. On the fictional cases, raw dense max-cosine to the NAVIGATOR corpus separates attack from benign by only 0.045 (attack mean 0.590 vs. benign mean 0.545), and five of six benign novel cases clear the 0.50 escalation floor—i.e. the raw signal would flag them. The BM25 lexical gate that preserves NAVIGATOR's benchmark precision makes this worse outside the benchmark, not better: because the gate discounts a score toward its floor whenever the query shares no attack keywords with the corpus—always the case for a genuinely novel vocabulary—the gated confidence of both attacks and benign calls collapses to roughly 0.38, below the 0.50 escalation floor, so NAVIGATOR simply goes *silent* on novel-domain attacks. This is a direct, structural consequence of the design, not an incidental miss: the mechanism that gives NAVIGATOR its low false-positive rate on the benchmark is the same mechanism that removes its signal off-benchmark. The one genuinely general component we built for NAVIGATOR—the observational/consequential tool classifier—correctly labeled all four novel tools (both novel consequential actions as CONSEQUENTIAL, both novel reads as OBSERVATIONAL) from verb-token structure alone. But that classifier is a *taxonomy, not a detector*: it identifies that an action is consequential without deciding whether it was *authorized*. Turning it into detection requires the pinned-intent authorized-flow layer, which we attempted and did not complete—a documented negative result, detailed in §6 above.

Concrete next steps. Two of these boundaries have specific, already-scoped remedies. For CHANNEL, a behavior-general destination detector for the taint/hard-block path—flagging a destination-shaped value (email, IBAN, URL, phone) in *any* tool's arguments, mirroring the fallback the provenance path already uses successfully—would extend the hard-block guarantee to novel tools without a name allowlist. For NAVIGATOR, a genuinely deterministic authorization layer that compares each consequential action against the pinned user intent—continuing from where the pinned-intent/authorized-flow attempts left off—would convert the already-correct taxonomy classifier into real detection that does not depend on corpus vocabulary. We frame this whole section as a

strength rather than an apology: it is the output of a deliberate, independent adversarial validation pass run *before* submission, specifically to surface these boundaries ourselves. Knowing precisely where each mechanism does and does not generalize—and having fixed the one confirmed false positive it found—is what distinguishes a rigorously evaluated system

B Extended Related Work Comparison

Table 1: Comparison of agent guardrails on five axes. "Boundary" indicates where in the agent lifecycle enforcement fires. "Mechanism" is the primary detection approach. "Ext." indicates whether the detection corpus / rule set is extensible without retraining. "Verdict" indicates how multiple detection signals are combined. AgentDojo ASR and InjecAgent Recall are the best numbers reported by the respective papers; *n/r* = not reported; *ext.* = experimental; † = estimated from paper description; ‡ = cited from the source paper as an upper reference point, not independently reproduced here. Lower ASR is better; higher Recall is better.

System	Boundary	Mechanism	Ext.?	Verdict	AgentDojo ASR ↓	InjecAgent Recall ↑
PromptGuard 2	input	DeBERTa classifier	no	threshold	7.5%	n/r
AlignmentCheck	CoT	LLM judge	no	threshold	2.89%	n/r
CodeShield	output	Semgrep+regex	yes (rules)	threshold	n/r	n/r
LlamaFirewall (combined)	I/CoT/O	3-layer spatial	partial	joint	1.75%	n/r
AgentSpec	plan	rule DSL	yes	all-pass	>10% blocked	n/r
DRIFT	plan	rule+isolator	partial	sequential	n/r	n/r
PlanGuard [11]	plan	isolated planner + verifier	no	hard-constraint	n/r	100%‡
IntentGuard [13]	plan	intent trace + origin	no	alert-first	n/r	n/r
Progent [18]	tool-call	progressive allowlist	yes (policy)	policy-match	1.0%‡	n/r
Task Shield	tool-call	LLM alignment	no	threshold	22%†	n/r
ShieldAgent	tool-call	rule circuits	yes (docs)	probabilistic	n/r	90.1%
PSG-Agent	tool-call	4-component	partial	cross-turn accum.	n/r	n/r
OpenClaw PRISM	tool-call	middleware checkpts	partial	sequential	n/r	n/r
Kavach (ours)	**tool-call**	**determ.+embed.**	**yes (rules+corpus)**	**veto**	**n/a**§	**100% (544/544 DS)**¶
CaMeL	IFC	dual-LLM + taint	no	provable	23% (sec)	n/r
FIDES	IFC	label propagation	no	provable	n/r	n/r
RTBAS	IFC	saliency screener	no	threshold	2% loss only	n/r
AGrail	adaptive	LLM + memory	yes (memory)	cooperative	3.8%–5%	n/r
TaskTracker	session	activation delta	no	binary	n/r	n/r
DeepContext	session	RNN over embeds	no	binary	n/r	n/r
Trajectory Guard	trajectory	Siamese RAE	no	score	n/r	n/r
garak [10]	input (test)	vuln. probes	yes (probes)	per-probe	n/r	n/r
PyRIT [16]	input (test)	red-team orch.	yes (attacks)	*n/a*	n/r	n/r
LLM-Guard [17]	input/output	detector suite	yes (scanners)	threshold	n/r	n/r
DKnownAI Guard	mixed	ensemble	partial	threshold	n/r	96.5%
Lakera Guard	input	classifier	no	threshold	n/r	n/r
AWS Bedrock	input	content filter	no	threshold	n/r	n/r

Veto verdict (Kavach): any single minister returning BLOCK vetoes the call unconditionally; no score averaging or learned aggregation. **Corpus extensibility** (Kavach): deterministic prefilters are extended by adding a structural rule; the cosine-triage corpus by adding a natural-language attack-pattern description, re-embedded without retraining. ¶DS = data-stealing subset; direct-harm subset is an open gap (§4.7). §Kavach deliberately reports per-mechanism trajectory analysis (§4.6) rather than a single AgentDojo ASR.

References

[1] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. 2024. Are you still on track!? Catching LLM Task Drift with Activations. arXiv:2406.00799 [cs.CR]

[2] Laksh Advani. 2026. Trajectory Guard – A Lightweight, Sequence-Aware Model for Real-Time Anomaly Detection in Agentic AI. AAAI 2026 Trustworthy Agentic AI Workshop.. arXiv:2601.00516 [cs.LG]

[3] Justin Albrethsen, Yash Datta, Kunal Kumar, and Sharath Rajasekar. 2026. DeepContext: Stateful Real-Time Detection of Multi-Turn Adversarial Intent Drift in LLMs. arXiv:2602.16935 [cs.AI]

[4] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. arXiv:2503.22738 [cs.CR]

[5] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, Stephanie Ding, Shengye Wan, Spencer Whitman, Lauren Deason, Nicholas Doucette, Abraham Montilla, Alekhya Gampa, Beto de Paola, Dominik Gabi, James Crnkovich, Jean-Christophe Testud, Kat He, Rashnil Chaturvedi, Wu Zhou, and Joshua Saxe. 2025. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. *arXiv preprint arXiv:2505.03574* (2025).

[6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2026. Securing AI Agents with Information-Flow Control. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. Introduces FIDES.. arXiv:2505.23643

[7] CrowdSec. 2026. CVE-2025-34291 Exploited in the Wild: LangFlow AI Framework Under Fire. https://www.crowdsec.net/vulntracking-report/cve-2025-34291. Active exploitation observed beginning 23 Jan 2026..

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2026. Defeating Prompt Injections by Design. In *Proceedings of the 2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. arXiv:2503.18813 [cs.CR]

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. arXiv:2406.13352 [cs.CR]

[10] Leon Derczynski, Erick Galinkin, Jeffrey Martin, Subho Majumdar, and Nanna Inie. 2024. garak: A Framework for Security Probing Large Language Models. NVIDIA. Software: https://github.com/NVIDIA/garak. garak = Generative AI Red-teaming and Assessment Kit.. arXiv:2406.11036 [cs.CL]

[11] Guangyu Gong and Zizhuang Deng. 2026. PlanGuard: Defending Agents against Indirect Prompt Injection via Planning-based Consistency Verification. *arXiv preprint arXiv:2604.10134* (2026). arXiv:2604.10134

[12] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. arXiv:2412.16682 [cs.CR]

[13] Mintong Kang, Chong Xiang, Sanjay Kariyappa, Chaowei Xiao, Bo Li, and Edward Suh. 2025. Mitigating Indirect Prompt Injection via Instruction-Following Intent Analysis. *arXiv preprint arXiv:2512.00966* (2025). arXiv:2512.00966

[14] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI Agents That Matter. arXiv:2407.01502 [cs.LG]

[15] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. 2025. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*. arXiv:2506.12104

[16] Microsoft AI Red Team. 2024. PyRIT: Python Risk Identification Tool for generative AI. https://github.com/Azure/PyRIT. Microsoft red-teaming framework..

[17] Protect AI. 2024. LLM Guard: a security toolkit for LLM interactions. https://github.com/protectai/llm-guard. Open-source input/output scanner suite..

[18] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025). arXiv:2504.11703

[19] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual Agent Security: A Policy for Every Purpose. arXiv:2501.17070 [cs.CR]

[20] Yaozu Wu, Jizhou Guo, Dongyuan Li, Henry Peng Zou, Wei-Chieh Huang, Yankai Chen, Zhen Wang, Weizhi Zhang, Yangning Li, Meng Zhang, Renhe Jiang, and Philip S. Yu. 2025. PSG-Agent: Personality-Aware Safety Guardrail for LLM-based Agents. arXiv:2509.23614 [cs.CR]

[21] Zhuowen Yuan, Zhaorun Chen, Zhen Xiang, Nathaniel D. Bastian, Seyyed Hadi Hashemi, Chaowei Xiao, Wenbo Guo, and Bo Li. 2026. ShieldNet: Network-Level Guardrails against Emerging Supply-Chain Injections in Agentic Systems. arXiv:2604.04426 [cs.AI]

[22] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. arXiv:2403.02691 [cs.CL]

[23] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. 2026. ClawGuard: A Runtime Security Framework for Tool-Augmented LLM Agents Against Indirect Prompt Injection. *arXiv preprint arXiv:2604.11790* (2026).

[24] Peter Yong Zhong, Siyuan Chen, and Others. 2025. RTBAS: Defending LLM Agents Against Prompt Injection and Privacy Leakage. arXiv:2502.08966 [cs.CR]

Use of Generative AI

Generative AI tools (Anthropic's Claude, via Claude Code) were used throughout this project's development: for implementing detection mechanisms based on the authors' architectural decisions, running and analyzing benchmark evaluations, drafting sections of this paper's text based on the authors' findings and direction, and assisting with code review and debugging. All experimental design choices, architectural decisions, evaluation methodology, and interpretation of results were directed and verified by the authors.